

Pre-requisites

1. FastAPI Framework Knowledge: [FastAPI Documentation](#)
2. Google Gemini API Familiarity: [Google Generative AI Documentation](#)
3. Streamlit for Frontend: [Streamlit Documentation](#)
4. Python Programming Proficiency: [Python Official Docs](#)
5. Environment Setup: pip install fastapi uvicorn streamlit python-docx fpdf Pillow requests google-generativeai python-dotenv

Project Workflow

Activity 1: Model Selection and System Architecture

- **1.1: Research and select the Model:** Gemini 1.5 Pro model from Google Generative AI for legal document generation.
- **1.2: Define architecture:**
 - Frontend: Streamlit for user input and document download.
 - Backend: FastAPI (main.py, routes.py) to handle API requests.
 - AI Integration: Gemini 1.5 API used inside ai_core/gemini_generator.py.
- **1.3: Set up development environment with .env file containing the Gemini API key.**

Activity 2: Core Functionalities Development

- **2.1: Implement core generation features:**
 - Legal document creation
 - Editable text preview
 - Format & export as .txt, .docx, and .pdf
- **2.2: Build FastAPI backend:**
 - Endpoint /generate to process POST requests
 - Accepts document_type, parties, terms, dates
 - Returns generated text from Gemini

Activity 3: main.py & API Logic

- **3.1: Create and configure main.py:**

- Initialize FastAPI app ○ Route / for home
 - Include router from routes.py
 - **3.2: Handle document generation logic in routes.py:**
 - Define Pydantic model DocumentRequest
 - Call GeminiDocumentGenerator.generate_document(...)
-

Activity 4: Frontend with Streamlit

- **4.1: Design user-friendly Streamlit interface:**
 - Input fields for all required data
 - Responsive layout with title and logo
 - **4.2: Dynamic document generation preview:**
 - Show styled editable output ○ Option to export as .txt, .docx, or .pdf
-

Activity 5: Deployment

- **5.1: Prepare deployment:**
 - Install production-ready dependencies
 - Set up Procfile, requirements.txt, or Dockerfile if needed
 - **5.2: Deploy application:**
 - Options: Render, Railway, Deta, Streamlit Community Cloud (frontend), Fly.io or VPS for backend
-

Activity 6: Conclusion

- **Finalize and test:**
 - Ensure end-to-end functionality ○ Test Gemini integration under different input combinations

MILESTONE 1: Model Selection and Architecture – LegalEase

In this milestone, we focus on selecting the optimal generative AI model for creating personalized and accurate legal documents using Google's Gemini model via the Google Generative AI SDK. We also define the system

architecture and set up the development environment to support document generation, editing, and exporting in various formats